

IF 2211 Strategi Algoritma
Ice Sliding Puzzle Solver

Laporan Tugas Kecil 3

Disusun untuk memenuhi tugas mata kuliah IF 2211 Strategi Algoritma pada
Semester 2 Tahun Akademik 2025/2026



Disusun oleh:

13524128 Safira Berlianti
13524135 Varistha Devi

**PROGRAM STUDI TEKNIK INFORMATIKA
SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA
INSTITUT TEKNOLOGI BANDUNG
2024**

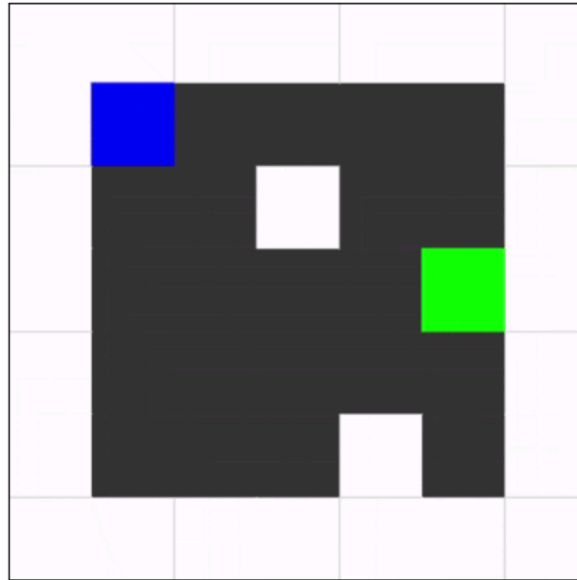
Daftar Isi

BAB 1 DESKRIPSI MASALAH.....	5
BAB 2 ALGORITMA PATHFINDING.....	7
2.1. Algoritma Uniform Cost Search (UCS).....	7
2.1.1. Deskripsi Algoritma.....	7
2.1.2. Langkah-Langkah Algoritma.....	7
2.1.3. Pseudocode.....	8
2.2. Algoritma Greedy Best-First Search (GBFS).....	9
2.2.1. Deskripsi Algoritma.....	9
2.2.2. Langkah-Langkah Algoritma.....	9
2.2.3. Pseudocode.....	10
2.3. Algoritma A* Search.....	11
2.3.1. Deskripsi Algoritma.....	11
2.3.2. Langkah-Langkah Algoritma.....	12
2.3.3. Pseudocode.....	12
2.4. Fungsi Pendukung.....	13
2.4.1. Fungsi slide(board, state, direction).....	13
2.4.2. Fungsi get_neighbors(board, state).....	14
2.4.3. Fungsi reconstruct_path(node).....	14
BAB 3 ANALISIS ALGORITMA.....	14
3.1. Definisi $f(n)$ dan $g(n)$ pada Algoritma.....	15
3.1.1. Definisi $g(n)$	15
3.1.2. Definisi $f(n)$ pada A*.....	15
3.2. Heuristik yang Digunakan.....	16
3.2.1. H1: Remaining Checkpoints and Goal (Manhattan).....	16
3.2.2. H2: Manhattan Distance ke Target Aktif Berikutnya.....	16
3.2.3. H3: Euclidean Distance ke Target Aktif Berikutnya.....	16
3.3. Apakah Heuristik yang Digunakan Admissible?.....	16
3.3.1. Admissibility H1.....	17
3.3.2. Admissibility H2.....	17
3.3.3. Admissibility H3.....	17
3.4. Perbandingan Teoritis Algoritma.....	18
3.4.1. Tabel Perbandingan.....	18
3.4.2. Analisis Perbandingan pada Ice Sliding Puzzle.....	18
3.4.3. Konsistensi Heuristik.....	19
BAB 4 SOURCE CODE.....	20
4.1. Folder algo_and_heu.....	20
4.1.1. heuristics.py.....	20
4.1.2. UCS.py.....	21
4.1.3. GBFS.py.....	22

4.1.4. AStar.py.....	23
4.2. Folder core.....	25
4.2.1. board.py.....	25
4.2.2. movement.py.....	25
4.2.3. node.py.....	27
4.2.4. state.py.....	28
4.3. Folder gui.....	29
4.3.1. Folder components.....	29
4.3.1.1. button.py.....	29
4.3.1.2. dropdown.py.....	30
4.3.1.3. label.py.....	33
4.3.1.4. panel.py.....	33
4.3.1.5. slider.py.....	34
4.3.2. Folder services.....	36
4.3.2.1. export_services.py.....	36
4.3.2.2. playback_service.py.....	38
4.3.2.3. solver_service.py.....	40
4.3.3. Folder utils.....	41
4.3.3.1. board_steps.py.....	41
4.3.3.2. timer.py.....	42
4.3.4. Folder views.....	43
4.3.4.1. board_view.py.....	43
4.3.4.2. playback_view.py.....	46
4.3.4.3. sidebar_view.py.....	47
4.3.4.4. statistics_view.py.....	49
4.3.5. app.py.....	50
4.3.6. colors.py.....	55
4.3.7. constants.py.....	56
4.4. Folder parser.....	56
4.4.1. parser.py.....	56
4.5. main.py.....	58
4.6. run_gui.py.....	66
BAB 5 PENGUJIAN.....	67
5.1. Pengujian.....	67
5.1.1. Testcase 1.....	67
5.1.2. Testcase 2.....	68
5.1.3. Testcase 3.....	68
5.1.4. Testcase 4.....	69
5.1.5. Testcase 5.....	69
5.1.6. Testcase 6.....	70
5.1.7. Testcase 7.....	70

5.2. Analisis Hasil.....	71
LAMPIRAN.....	72

BAB 1 DESKRIPSI MASALAH



Visualisasi Permainan Ice Sliding Puzzle Solver

Ice Sliding Puzzle adalah permainan logika di mana pemain harus menggerakkan karakter dari titik awal menuju titik keluar di atas permukaan es yang licin. Pin (dalam visualisasi warna biru) hanya dapat bergerak secara horizontal atau vertikal, namun karena permukaan licin, karakter tidak akan berhenti bergerak sampai menabrak dinding atau rintangan (dalam visualisasi warna putih). Tantangan utama permainan ini terletak pada bagaimana menghitung urutan gerakan yang tepat agar pin dapat mencapai titik keluar dengan cost minimal, sembari melewati checkpoint secara berurutan (dari angka 0, 1, 2, ...) tanpa melanggar aturan permainan.

Komponen-komponen penting dalam permainan ini terdiri dari papan, pin, checkpoint, rintangan, lava, titik keluar, dan cost traversal. Papan merupakan area permainan yang terdiri atas sejumlah cell/tile, yaitu unit terkecil yang dapat ditempati atau dilewati oleh pin. Setiap cell memiliki cost traversal yang merepresentasikan biaya untuk melewati cell tersebut. Pin adalah karakter yang digerakkan pemain, dilambangkan dengan huruf 'Z' pada konfigurasi awal yang hanya dapat bergerak horizontal (kiri/kanan) atau vertikal (atas/bawah) dan akan terus meluncur pada permukaan es hingga menabrak rintangan atau keluar dari papan.

Checkpoint adalah angka-angka (0, 1, 2, ..., 9) yang harus dilewati oleh pin secara berurutan. Apabila pin melewati checkpoint yang tidak sesuai urutan (misalnya melewati angka 2 sebelum melewati angka 1), maka permainan akan berakhir dengan game over. Setelah checkpoint dilewati, tile tersebut menjadi tile biasa dan dapat dilewati kembali tanpa batasan urutan. Rintangan, dilambangkan dengan huruf 'X', adalah tile yang menghentikan pergerakan pin. Pin akan berhenti tepat sebelum rintangan ketika melakukan gerakan sliding. Lava, dilambangkan dengan huruf 'L', adalah tile berbahaya yang menyebabkan game over apabila dilewati oleh pin, meskipun pin tidak berhenti di tile tersebut.

Titik keluar, dilambangkan dengan huruf 'O', adalah tujuan akhir yang harus dicapai pin. Pin harus berhenti tepat di tile 'O' setelah semua checkpoint (0, 1, 2, ..., n) telah dilewati secara berurutan agar permainan dianggap selesai dengan sukses. Tile 'O' tidak menghentikan pergerakan pin—artinya, pin dapat melewati tile 'O' tanpa berhenti apabila belum memenuhi syarat (checkpoint belum lengkap). Selain itu, apabila pin bergerak menuju sisi kanan/atas/kiri/bawah papan tetapi tidak terdapat rintangan pada sisi tersebut, maka pin akan keluar dari papan dan mengalami game over.

Cost traversal adalah biaya yang dikenakan ketika pin melewati suatu tile. Cost movement dihitung berdasarkan jumlah cost seluruh tile yang dilalui selama satu gerakan sliding, bukan hanya tile tempat pin berhenti. Tile posisi awal pin pada awal gerakan tidak dihitung, namun tile tujuan akhir gerakan tetap dihitung karena tile tersebut dilalui dan menjadi posisi berhenti pin. Sebagai contoh, jika pin melakukan gerakan ke kanan dan melewati 3 tile dengan cost 2, 5, dan 4 sebelum berhenti, maka cost gerakan tersebut adalah $2 + 5 + 4 = 11$. Total cost solusi adalah penjumlahan seluruh cost movement dari setiap gerakan dalam solusi. Tile 'X' (rintangan) dan 'L' (lava) tetap memiliki cost pada input, tetapi tile 'X' tidak pernah dilewati karena merupakan rintangan, dan tile 'L' menyebabkan game over jika dilewati, sehingga jalur yang melewati 'L' dianggap tidak valid.

Pada tugas kecil kali ini, mahasiswa ditugaskan untuk membuat program dalam bahasa Java, Python, C++, C, atau Go yang mengimplementasikan algoritma pathfinding Uniform Cost Search (UCS), Greedy Best First Search (GBFS), dan A* dalam menyelesaikan permainan Ice Sliding Puzzle. Program harus mampu menemukan solusi dengan biaya (cost) minimal atau solusi pertama yang valid, tergantung pada algoritma yang digunakan. Algoritma pathfinding minimal menggunakan satu heuristik yang ditentukan sendiri dan dijalankan secara terpisah. Algoritma dan heuristik yang digunakan ditentukan berdasarkan input pengguna.

BAB 2 ALGORITMA PATHFINDING

Pada tugas kecil ini, diimplementasikan tiga algoritma pathfinding untuk menyelesaikan permainan Ice Sliding Puzzle: Uniform Cost Search (UCS), Greedy Best-First Search (GBFS), dan A* Search. Ketiga algoritma ini diimplementasikan dengan memanfaatkan struktur data priority queue untuk memilih node yang akan dieksplorasi berikutnya berdasarkan fungsi evaluasi tertentu. Berikut adalah penjelasan detail langkah-langkah dari masing-masing algoritma.

2.1. Algoritma Uniform Cost Search (UCS)

2.1.1. Deskripsi Algoritma

Uniform Cost Search (UCS) adalah algoritma pencarian uninformed search yang menjamin menemukan solusi dengan biaya minimum. UCS bekerja dengan mengeksplorasi node berdasarkan biaya kumulatif dari node awal ke node saat ini, tanpa mempertimbangkan estimasi jarak ke goal.

Dalam konteks Ice Sliding Puzzle, biaya gerakan dihitung dari jumlah cost tile yang dilalui selama satu gerakan sliding. UCS akan selalu memilih node dengan total cost terendah untuk dieksplorasi terlebih dahulu menggunakan priority queue.

Algoritma ini diimplementasikan dengan menggunakan referensi dari:

- https://ursinus-cs477-f2023.github.io/CoursePage/ClassExercises/Week3_PrioritySearch/
- <https://medium.com/@kapil.lanjewar.100/8-puzzle-problem-or-fixing-the-grid-matrix-using-uniform-cost-search-ucs-f7de8e96c140>

2.1.2. Langkah-Langkah Algoritma

1. Inisialisasi:
 - Buat state awal dengan posisi pin (Z) dan last_num = -1 (belum melewati checkpoint apapun).
 - Buat node awal dengan cost = 0 dan parent = None.
 - Masukkan node awal ke dalam priority queue dengan prioritas berdasarkan cost kumulatif.
 - Inisialisasi visited sebagai set kosong untuk menyimpan state yang sudah dieksplorasi.
 - Inisialisasi costSoFar sebagai dictionary untuk menyimpan cost terkecil yang ditemukan untuk mencapai setiap state.
2. Loop Utama:
 - Selama priority queue tidak kosong:
 - Pop node dengan cost kumulatif terendah dari priority queue.
 - Increment counter iterations.
 - Jika state dari node tersebut sudah ada di visited, skip (sudah dieksplorasi sebelumnya).
 - Tandai state sebagai visited.
3. Goal Test:

- Periksa apakah state saat ini adalah goal state:
 - Pin berada di tile 'O'
 - Semua checkpoint (0, 1, 2, ..., max_num) telah dilewati secara berurutan.
 - Jika YA, rekonstruksi path dari node ini dengan menelusuri parent pointer hingga node awal, kemudian return solusi.
4. Ekspansi Neighbors:
- Dapatkan semua state yang dapat dicapai dari state saat ini dengan mencoba keempat arah (UP, DOWN, LEFT, RIGHT).
 - Untuk setiap neighbor:
 - Hitung new_cost = cost node saat ini + movement_cost (cost sliding).
 - Jika neighbor sudah visited, skip.
 - Jika new_cost lebih kecil dari cost sebelumnya yang tercatat di costSoFar (atau belum pernah tercatat), update costSoFar dan masukkan neighbor ke priority queue.
5. Terminasi:
- Jika priority queue habis tanpa menemukan goal, return tidak ada solusi.

2.1.3. Pseudocode

```

function UCS(board):
  startState ← State(board.start.x, board.start.y, last_num=-1)
  startNode ← Node(state=startState, parent=null, move=null, cost=0)

  frontier ← PriorityQueue() // berdasarkan cost
  frontier.push((0, startNode))

  visited ← Set()
  costSoFar ← Dictionary()
  costSoFar[startState] ← 0

  iterations ← 0

  while frontier is not empty:
    (currentCost, currentNode) ← frontier.pop()
    iterations ← iterations + 1

    if currentNode.state in visited:
      continue

    visited.add(currentNode.state)

    if board.is_goal(currentNode.state):
      return reconstruct_path(currentNode), currentNode.cost, iterations

```



```

neighbors ← get_neighbors(board, currentNode.state)

for (dirChar, nextState, moveCost) in neighbors:
    newCost ← currentNode.cost + moveCost

    if nextState in visited:
        continue

    if nextState not in costSoFar or newCost < costSoFar[nextState]:
        costSoFar[nextState] ← newCost
        newNode ← Node(state=nextState, parent=currentNode,
                        move=dirChar, cost=newCost)
        frontier.push((newCost, newNode))

return no_solution

```

2.2. Algoritma Greedy Best-First Search (GBFS)

2.2.1. Deskripsi Algoritma

Greedy Best-First Search (GBFS) adalah algoritma pencarian informed search yang hanya menggunakan fungsi heuristik $h(n)$ untuk memilih node berikutnya yang akan dieksplorasi. GBFS bekerja secara greedy dengan selalu memilih node yang paling dekat dengan goal berdasarkan estimasi heuristik, tanpa mempertimbangkan biaya aktual yang telah dikeluarkan untuk mencapai node tersebut.

GBFS cenderung lebih cepat dibanding UCS karena fokus langsung menuju goal, tetapi tidak menjamin menemukan solusi optimal karena mengabaikan biaya aktual perjalanan.

Algoritma ini diimplementasikan dengan menggunakan referensi dari:

- https://ursinus-cs477-f2023.github.io/CoursePage/ClassExercises/Week3_PrioritySearch/
- https://medium.com/@Andrew_D/ai-search-techniques-for-sliding-8-tile-puzzle-6ba13490149
- <https://jpeckham.com/projects/8-puzzle-solver/>

2.2.2. Langkah-Langkah Algoritma

1. Inisialisasi:
 - Buat state awal dengan posisi pin (Z) dan last_num = -1.
 - Hitung nilai heuristik untuk state awal menggunakan fungsi heuristik yang dipilih (H1/H2/H3).
 - Buat node awal dengan cost = 0 dan heuristic = $h(\text{startState})$.
 - Masukkan node awal ke priority queue dengan prioritas berdasarkan nilai heuristik (bukan cost).

- Inisialisasi visited sebagai set untuk menyimpan state yang sudah dieksplorasi.
- Inisialisasi frontier_set untuk tracking state yang ada di dalam frontier (optimisasi).
- 2. Loop Utama:
 - Selama priority queue tidak kosong:
 - Pop node dengan nilai heuristik terendah dari priority queue.
 - Increment counter iterations.
 - Hapus state dari frontier_set.
 - Jika state sudah ada di visited, skip.
 - Tandai state sebagai visited.
- 3. Goal Test:
 - Periksa apakah state saat ini adalah goal state (pin di 'O' dan semua checkpoint dilewati).
 - Jika YA, rekonstruksi path dan return solusi.
- 4. Ekspansi Neighbors:
 - Dapatkan semua state yang dapat dicapai dari state saat ini.
 - Untuk setiap neighbor:
 - Jika neighbor sudah visited atau sudah ada di frontier_set, skip (hindari duplikasi).
 - Hitung cost aktual newCost untuk tracking (meskipun tidak digunakan untuk prioritas).
 - Hitung nilai heuristik h(neighbor).
 - Buat node baru dan masukkan ke priority queue dengan prioritas h(neighbor).
 - Tambahkan neighbor ke frontier_set.
- 5. Terminasi:
 - Jika priority queue habis tanpa menemukan goal, return tidak ada solusi.

2.2.3. Pseudocode

```
function GBFS(board, heuristic):
  startState ← State(board.start.x, board.start.y, last_num=-1)
  heuVal ← heuristic(startState, board)
  startNode ← Node(state=startState, parent=null, move=null,
                  cost=0, heuristic=heuVal)

  frontier ← PriorityQueue() // berdasarkan heuristic
  frontier.push((heuVal, startNode))
  frontier_set ← Set()
  frontier_set.add(startState)

  visited ← Set()
  iterations ← 0

  while frontier is not empty:
    (heuVal, currentNode) ← frontier.pop()
```

```

iterations ← iterations + 1
frontier_set.remove(currentNode.state)

if currentNode.state in visited:
    continue

visited.add(currentNode.state)

if board.is_goal(currentNode.state):
    return reconstruct_path(currentNode), currentNode.cost, iterations

neighbors ← get_neighbors(board, currentNode.state)

for (dirChar, nextState, moveCost) in neighbors:
    if nextState in visited or nextState in frontier_set:
        continue

    newCost ← currentNode.cost + moveCost
    heuVal ← heuristic(nextState, board)
    newNode ← Node(state=nextState, parent=currentNode,
                    move=dirChar, cost=newCost, heuristic=heuVal)

    frontier.push((heuVal, newNode))
    frontier_set.add(nextState)

return no_solution

```

2.3. Algoritma A* Search

2.3.1. Deskripsi Algoritma

A* Search adalah algoritma pencarian informed search yang menggabungkan keunggulan UCS dan GBFS. A* menggunakan fungsi evaluasi $f(n) = g(n) + h(n)$, di mana:

- $g(n)$ = biaya aktual dari state awal ke node n (seperti UCS)
- $h(n)$ = estimasi biaya dari node n ke goal (seperti GBFS)

Dengan mempertimbangkan kedua komponen tersebut, A* mampu menemukan solusi optimal (jika heuristik admissible) dengan efisiensi lebih tinggi dibanding UCS karena menggunakan informasi heuristik untuk memangkas ruang pencarian yang tidak menjanjikan.

Algoritma ini diimplementasikan dengan menggunakan referensi dari:

- <https://medium.com/@kamilmatejuk/exploring-algorithms-heuristic-analysis-of-a-pathfinding-for-puzzle15-2a9dfda87e1f>
- <https://medium.com/@ranjanasattavan143/the-8-puzzle-problem-a-complete-ai-formulation-0071514b9589>

2.3.2. Langkah-Langkah Algoritma

1. Inisialisasi:
 - Buat state awal dengan posisi pin (Z) dan last_num = -1.
 - Hitung nilai heuristik untuk state awal: $h(\text{startState})$.
 - Buat node awal dengan cost = 0, heuristic = $h(\text{startState})$, dan $f_score = g + h = 0 + h(\text{startState})$.
 - Masukkan node awal ke priority queue dengan prioritas berdasarkan f_score .
 - Inisialisasi visited sebagai dictionary untuk menyimpan minimum cost yang ditemukan untuk mencapai setiap state.
2. Loop Utama:
 - Selama priority queue tidak kosong:
 - Pop node dengan f_score terendah dari priority queue.
 - Increment counter iterations.
 - Jika cost node saat ini lebih besar dari cost terbaik yang tercatat di visited[state], skip (sudah ada path yang lebih baik).
3. Goal Test:
 - Periksa apakah state saat ini adalah goal state.
 - Jika YA, rekonstruksi path dan return solusi.
4. Ekspansi Neighbors:
 - Dapatkan semua state yang dapat dicapai dari state saat ini.
 - Untuk setiap neighbor:
 - Hitung $\text{newCost} = g(\text{current}) + \text{moveCost}$.
 - Jika neighbor sudah ada di visited dengan cost lebih kecil atau sama, skip (path sekarang tidak lebih baik).
 - Update $\text{visited}[\text{neighbor}] = \text{newCost}$.
 - Hitung $h(\text{neighbor})$ menggunakan fungsi heuristik.
 - Hitung $f_score = \text{newCost} + h(\text{neighbor})$.
 - Buat node baru dan masukkan ke priority queue dengan prioritas f_score .
5. Terminasi:
 - Jika priority queue habis tanpa menemukan goal, return tidak ada solusi.

2.3.3. Pseudocode

```
function AStar(board, heuristic):  
    startState ← State(board.start.x, board.start.y, last_num=-1)  
    startNode ← Node(state=startState, parent=null, move=null,  
                     cost=0, heuristic=heuristic(startState, board))
```

```

frontier ← PriorityQueue() // berdasarkan f_score
frontier.push((startNode.f_score, startNode))

visited ← Dictionary() // menyimpan min cost per state
visited[startState] ← 0

iterations ← 0

while frontier is not empty:
    (f_score, currentNode) ← frontier.pop()
    currentState ← currentNode.state
    iterations ← iterations + 1

    if currentNode.cost > visited[currentState]:
        continue

    if board.is_goal(currentState):
        return reconstruct_path(currentNode), currentNode.cost, iterations

    neighbors ← get_neighbors(board, currentState)

    for (dirChar, nextState, moveCost) in neighbors:
        newCost ← currentNode.cost + moveCost

        if nextState in visited and newCost >= visited[nextState]:
            continue

        visited[nextState] ← newCost
        heuVal ← heuristic(nextState, board)
        newNode ← Node(state=nextState, parent=currentNode,
                        move=dirChar, cost=newCost, heuristic=heuVal)

        frontier.push((newNode.f_score, newNode))

return no_solution

```

2.4. Fungsi Pendukung

2.4.1. Fungsi slide(board, state, direction)

Fungsi ini mensimulasikan pergerakan pin dalam satu arah hingga berhenti karena menabrak rintangan atau keluar dari papan.

Langkah-langkah:

1. Ambil delta posisi (dx, dy) berdasarkan arah gerakan.
2. Inisialisasi total_movement_cost = 0.
3. Loop hingga pin berhenti:
 - Hitung posisi berikutnya (nx, ny) = (x + dx, y + dy).
 - Jika keluar dari papan → return None (game over).
 - Jika tile = 'L' (lava) → return None (game over).
 - Jika tile = 'X' (rintangan) → break (berhenti sebelum rintangan).
 - Jika tile = angka (checkpoint):
 - Jika angka = last_num + 1 → update last_num (checkpoint valid).
 - Jika angka ≤ last_num → checkpoint sudah dilewati, lanjutkan.
 - Jika angka > last_num + 1 → return None (melanggar urutan).
 - Tambahkan costs[nx][ny] ke total_movement_cost.
 - Update posisi (x, y) = (nx, ny).
4. Jika tidak ada pergerakan sama sekali → return None.
5. Return (new_state, total_movement_cost).

2.4.2. Fungsi get_neighbors(board, state)

Fungsi ini menghasilkan semua state yang dapat dicapai dari state saat ini dengan mencoba keempat arah gerakan.

Langkah-langkah:

1. Inisialisasi list kosong neighbors.
2. Untuk setiap arah (UP, DOWN, LEFT, RIGHT):
 - Panggil slide(board, state, direction).
 - Jika hasilnya None → skip (gerakan invalid).
 - Jika pin tidak bergerak (posisi sama) → skip.
 - Tambahkan (direction_char, next_state, movement_cost) ke neighbors.
3. Return neighbors.

2.4.3. Fungsi reconstruct_path(node)

Fungsi ini merekonstruksi path dari goal node dengan menelusuri parent pointer hingga node awal.

Langkah-langkah:

1. Inisialisasi list kosong path.
2. Mulai dari goal node, loop selama node memiliki parent:
 - Append node.move ke path.
 - Update node = node.parent.
3. Reverse path (karena dibangun dari goal ke start).
4. Join array menjadi string dan return.

BAB 3 ANALISIS ALGORITMA

3.1. Definisi $f(n)$ dan $g(n)$ pada Algoritma

3.1.1. Definisi $g(n)$

Pada ketiga algoritma yang diimplementasikan, $g(n)$ merepresentasikan biaya aktual kumulatif dari state awal hingga state n . Biaya ini dihitung berdasarkan cost tile yang dilalui selama pergerakan sliding.

Pada kode, $g(n)$ disimpan sebagai atribut cost pada setiap node. Ketika pin bergerak (sliding), biaya dihitung sebagai penjumlahan `board.costs[nx][ny]` dari setiap tile yang dilintasi dalam satu gerakan, diimplementasikan dalam fungsi `slide()` di `movement.py`:

```
total_movement_cost += board.costs[nx][ny]
```

Jika pin bergerak melewati 3 tile dengan cost masing-masing 1, 2, dan 3, maka `moveCost = 6`. Biaya kumulatif node berikutnya:

$$g(n') = g(n) + \text{moveCost}$$

Pada UCS, hanya $g(n)$ yang digunakan sebagai fungsi prioritas. Pada GBFS, $g(n)$ tetap dihitung untuk tracking, tetapi tidak digunakan sebagai prioritas. Pada A*, $g(n)$ merupakan komponen pertama dari fungsi evaluasi.

3.1.2. Definisi $f(n)$ pada A*

Pada A* Search, fungsi evaluasi $f(n)$ didefinisikan sebagai:

$$f(n) = g(n) + h(n)$$

Di mana:

- $g(n)$ = biaya aktual kumulatif dari start ke node n (atribut cost pada Node)
- $h(n)$ = nilai heuristik, yaitu estimasi biaya dari node n ke goal (atribut heuristic pada Node)

Pada kode, $f(n)$ diimplementasikan sebagai property `f_score` di `node.py`:

```
@property
def f_score(self):
    return self.cost + self.heuristic
```

Node dengan $f(n)$ terkecil dipilih terlebih dahulu dari priority queue, sehingga A* menyeimbangkan antara biaya yang sudah dikeluarkan dan estimasi sisa biaya menuju goal.

Pada GBFS, fungsi prioritasnya hanya $h(n)$ (bukan $f(n)$), sehingga algoritma ini sepenuhnya greedy dan mengabaikan $g(n)$ dalam pengambilan keputusan.

3.2. Heuristik yang Digunakan

Tiga heuristik diimplementasikan dalam heuristics.py:

3.2.1. H1: Remaining Checkpoints and Goal (Manhattan)

H1 menghitung total Manhattan distance dari posisi saat ini ke semua checkpoint yang belum dikunjungi secara berurutan, lalu dari checkpoint terakhir ke goal:

$$h_1(n) = \sum_{i=\text{last_num}+1}^{\text{max_num}} d_M(\text{pos}_{i-1}, \text{cpi}) + d_M(\text{cpmax_num}, \text{goal})$$

Di mana $d_M(a, b) = |x_a - x_b| + |y_a - y_b|$ adalah Manhattan distance, dan pos_0 adalah posisi pin saat ini.

3.2.2. H2: Manhattan Distance ke Target Aktif Berikutnya

H2 menghitung Manhattan distance hanya ke satu target berikutnya, yakni checkpoint berikutnya jika masih ada, atau goal jika semua checkpoint sudah dilewati:

$$h_2(n) = d_M(\text{posisi saat ini}, \text{target aktif})$$

3.2.3. H3: Euclidean Distance ke Target Aktif Berikutnya

H3 identik dengan H2 tetapi menggunakan Euclidean distance:

$$h_3(n) = \sqrt{(x - x_{\text{target}})^2 + (y - y_{\text{target}})^2}$$

3.3. Apakah Heuristik yang Digunakan Admissible?

Sebuah heuristik $h(n)$ disebut admissible jika ia tidak pernah melebihi biaya aktual dari node n ke goal:

$$h(n) \leq h^*(n) \quad \forall n$$

di mana $h^*(n)$ adalah biaya optimal sesungguhnya dari n ke goal.

Kunci analisis: pada puzzle ini, satu gerakan sliding dapat melewati lebih dari satu tile, dan setiap tile memiliki cost ≥ 1 . Karena itu, jarak tempuh minimum untuk berpindah antara dua posisi (dalam satuan cost tile) adalah minimal sama dengan jarak Manhattan atau Euclidean antarposisi tersebut — dengan asumsi cost tile ≥ 1 .

3.3.1. Admissibility H1

H1 menghitung total Manhattan distance dari posisi saat ini melalui semua checkpoint yang tersisa hingga goal, dievaluasi seolah-olah perjalanan antar checkpoint dilakukan dalam garis lurus tanpa hambatan.

H1 adalah admissible, dengan alasan:

- Manhattan distance antara dua titik adalah batas bawah dari jarak sebenarnya pada grid dengan gerakan sliding, karena gerakan aktual tidak bisa lebih pendek dari jarak Manhattan (meskipun mekanisme sliding bisa "melampaui" target, pin tetap harus menempuh jarak minimal sebesar Manhattan distance dalam cost tile).
- H1 mengasumsikan semua tile memiliki cost = 1 dan tidak ada rintangan, sehingga nilainya selalu $\leq$ biaya aktual.
- H1 juga mempertimbangkan semua checkpoint yang tersisa, bukan hanya satu, sehingga estimasinya lebih ketat dibandingkan dengan H2 dan H3.

3.3.2. Admissibility H2

H2 hanya mempertimbangkan jarak ke satu target berikutnya, mengabaikan checkpoint-checkpoint setelahnya.

H2 adalah admissible, karena:

- Manhattan distance ke target aktif $\leq$ biaya aktual mencapai target tersebut $\leq$ biaya aktual mencapai goal.
- Dengan hanya menghitung sebagian dari perjalanan, H2 justru menghasilkan estimasi yang lebih rendah dibanding H1, sehingga lebih "aman" dari sisi admissibility tetapi kurang informatif.

3.3.3. Admissibility H3

H3 menggunakan Euclidean distance ke target aktif berikutnya.

H3 adalah admissible, karena:

- Euclidean distance $\leq$ Manhattan distance untuk dua titik yang sama ($d_E \leq d_M$), sehingga jika H2 admissible maka H3 juga admissible.
- Euclidean distance merupakan batas bawah yang lebih longgar (lebih kecil) dibanding Manhattan distance, sehingga kurang informatif tetapi tetap tidak melebihi biaya aktual.

3.4. Perbandingan Teoritis Algoritma

3.4.1. Tabel Perbandingan

Aspek	UCS	GBFS	A*
Fungsi prioritas	$g(n)$	$h(n)$	$f(n) = g(n) + h(n)$
Completeness	Ya (jika $cost > 0$)	Tidak dijamin	Ya (jika $cost > 0$)
Optimality	Ya	Tidak dijamin	Ya (jika h admissible)
Jenis pencarian	Uninformed	Informed	Informed
Kompleksitas waktu	$O(b^{(C^*/\epsilon)})$	$O(b^m)$	$O(b^d)$
Kompleksitas ruang	$O(b^{(C^*/\epsilon)})$	$O(b^m)$	$O(b^d)$

Note: b = branching factor, d = kedalaman solusi optimal, m = kedalaman maksimum, C = biaya optimal, ϵ = biaya minimum satu langkah.

3.4.2. Analisis Perbandingan pada Ice Sliding Puzzle

Uniform Cost Search (UCS)

UCS mengeksplorasi semua node yang memiliki cost kumulatif $\leq$ cost solusi optimal. Tanpa informasi heuristik, UCS "buta" terhadap posisi goal dan akan mengeksplorasi semua arah secara merata. Pada puzzle dengan papan besar dan banyak checkpoint, UCS berpotensi mengeksplorasi jauh lebih banyak node dibandingkan dengan A*, karena tidak ada panduan menuju goal. Meski demikian, UCS selalu menjamin solusi dengan cost minimum.

Greedy Best-First Search (GBFS)

GBFS mengeksplorasi dengan cepat menuju goal menggunakan heuristik, sehingga biasanya memerlukan lebih sedikit iterasi dibandingkan dengan UCS. Namun, karena mengabaikan $g(n)$, GBFS dapat terjebak dalam path dengan cost besar yang heuristiknya rendah, atau terlewatkan oleh adanya rintangan dan checkpoint. Pada puzzle ini, mekanisme sliding dan urutan checkpoint membuat GBFS rentan menemukan solusi suboptimal — path yang ditemukan lebih dulu secara heuristik belum tentu path termurah.

A* Search

A* menggabungkan keunggulan kedua algoritma: menjamin optimalitas seperti UCS, sekaligus menggunakan heuristik untuk memangkas pencarian seperti GBFS. Dengan heuristik yang lebih informatif ($H1 > H2 > H3$ dalam hal informativeness), A* akan mengeksplorasi lebih sedikit node. Semakin ketat heuristik (semakin mendekati h^*), semakin efisien A*.

Pada puzzle ini, H1 paling dominan secara informatif karena mempertimbangkan seluruh sisa perjalanan (semua checkpoint + goal), bukan hanya satu langkah ke depan. Secara teoritis, A*-H1 diharapkan mengeksplorasi node paling sedikit di antara semua kombinasi.

Perbandingan informativeness heuristik:

$$h_3(n) \leq h_2(n) \leq h_1(n) \leq h^*(n)$$

Semakin besar nilai heuristik (selama masih admissible), semakin sedikit node yang perlu dieksplorasi oleh A*, dan semakin mendekati perilaku optimal. H1 adalah heuristik paling kuat di antara ketiganya.

3.4.3. Konsistensi Heuristik

Selain admissible, heuristik yang baik untuk A* sebaiknya juga consistent (monotone):

$$h(n) \leq c(n, n') + h(n')$$

di mana $c(n, n')$ adalah biaya aktual dari n ke n' . Heuristik yang konsisten menjamin bahwa A* tidak perlu memproses ulang node yang sudah dieksplorasi.

H1 dan H2 (Manhattan distance) bersifat konsisten pada grid standar karena Manhattan distance memenuhi triangle inequality. Namun, pada puzzle ini gerakan sliding melewati lebih dari satu tile sekaligus, yang berarti heuristik berbasis jarak langsung mungkin kurang presisi untuk konsistensi yang ketat. Secara praktis, admissibility sudah cukup untuk menjamin optimalitas A*.

H3 (Euclidean distance) juga konsisten secara umum, karena Euclidean distance memenuhi triangle inequality di ruang kontinu.

BAB 4 SOURCE CODE

4.1. Folder algo_and_heu

4.1.1. heuristics.py

```
import math

def getHeuristic(name):
    heuristics = {
        "H1" : remCheckpointsAndGoal,
        "H2" : manhattanToNextActiveTar,
        "H3" : euclideanToNextActiveTar,
    }
    return heuristics.get(name)

#total manhattan distance untuk semua checkpoint yang tersisa ditambah goal
def remCheckpointsAndGoal(state, board):
    totDist = 0
    currentPos = (state.x, state.y)
    #jarak ke next checkpoint
    for num in range(state.last_num + 1, board.max_num + 1):
        nextPos = board.checkpointPos[num]
        totDist += abs(currentPos[0] - nextPos[0]) + abs(currentPos[1] -
nextPos[1])
        currentPos = nextPos #update position buat checkpoint berikutnya
    #jarak checkpoint terakhir ke goal
    totDist += abs(currentPos[0] - board.goalPos[0]) + abs(currentPos[1] -
board.goalPos[1])
    return totDist

#ngitung manhattan distance ke target aktif (bisa checkpoint berikutnya
atau goal)
def manhattanToNextActiveTar(state, board):
    nextNum = state.last_num + 1
    #kalo ada checkpoint yang masih belum keambil
    if nextNum <= board.max_num:
        targetX, targetY = board.checkpointPos[nextNum]
    else: #kalau udah keambil semua, change target to goal
```

```

        targetX, targetY = board.goalPos
    return abs(state.x - targetX) + abs(state.y - targetY)

#hitung euclidean distance ke target aktif
def euclideanToNextActiveTar(state, board):
    nextNum = state.last_num + 1
    if nextNum <= board.max_num:
        targetX, targetY = board.checkpointPos[nextNum]
    else:
        targetX, targetY = board.goalPos
    return math.sqrt((state.x - targetX)**2 + (state.y - targetY)**2)

```

4.1.2. UCS.py

```

from heapq import heappush, heappop
from src.core.state import State
from src.core.node import Node
from src.core.movement import get_neighbors

def ucs(board):
    #init starting state dan node
    startState = State(x=board.start.x, y=board.start.y, last_num=-1)
    startNode = Node(state=startState, parent=None, move=None, cost=0)
    #menggunakan priority queue yang menyimpan (cost, node). karena
    #menggunakan heapq, cost paling rendah akan dipop lebih dahulu
    frontier = []
    heappush(frontier, (0, startNode))
    visited = set()
    #track best cost untuk mencapai tiap state (buat update frontier)
    costSoFar = {startState:0}
    iterations = 0
    while frontier:
        #pop state dengan cost terendah
        currentCost, currentNode = heappop(frontier)
        iterations += 1
        #skip kalo udah visited
        if currentNode.state in visited: continue
        #kalau nggak, tandain visited
        visited.add(currentNode.state)
        #check goal, apakah udah sampe 'O' dan melewati semua checkpoint
        if board.isGoal(currentNode.state):
            print("Jarak terpendek adalah", currentCost)
            return currentCost
    return None

```

```

        if board.is_goal(currentNode.state):
            return{'solution': Node.reconstruct_path(currentNode), 'cost':
currentNode.cost, 'iterations': iterations, 'found': True}
        #expand neighbors, coba keempat arah
        neighbors = get_neighbors(board, currentNode.state)
        for dirChar, nextState, moveCost in neighbors:
            #hitung cost kumulatif untuk reach next state
            newCost = currentNode.cost + moveCost
            #skip kalau udah visited
            if nextState in visited: continue
            #hanya add kalau kita nemuin path yang lebih murah
            if nextState not in costSoFar or newCost <
costSoFar[nextState]:
                costSoFar[nextState] = newCost
                #bikin node baru
                newNode = Node(state=nextState, parent=currentNode,
move=dirChar, cost=newCost)
                heappush(frontier, (newCost, newNode))
        #kalau frontier habis tanpa menemukan goal, berarti nggak ada solusi
        return {'solution': None, 'cost': 0, 'iterations': iterations,
'found': False}

```

4.1.3. GBFS.py

```

from heapq import heappush, heappop
from src.core.state import State
from src.core.node import Node
from src.core.movement import get_neighbors

def gbfs(board, heuristics):
    #init starting state dan node
    startState = State(board.start.x, board.start.y, last_num=-1)
    heuVal = heuristics(startState, board)
    startNode = Node(state=startState, parent=None, move=None, cost=0,
heuristic=heuVal)
    #priority queue yang menyimpan (heuristic_value, node)
    frontier = []
    frontier_set = set()
    heappush(frontier, (heuVal, startNode))
    frontier_set.add(startState)

```

```

visited = set()
iters = 0
while frontier:
    #pop node dengan heuristic value terendah
    heuVal, curNode = heappop(frontier)
    iters += 1
    frontier_set.discard(curNode.state)
    if curNode.state in visited: continue
    #mark visited
    visited.add(curNode.state)
    #goal test
    if board.is_goal(curNode.state):
        return {'solution': Node.reconstruct_path(curNode), 'cost':
curNode.cost, 'iterations': iters, 'found': True}
    #expand neighbors
    neighbors = get_neighbors(board, curNode.state)
    for dirChar, nextState, moveCost in neighbors:
        if nextState in visited or nextState in frontier_set: continue
        #hitung cost aktual untuk tracking
        newCost = curNode.cost + moveCost
        #hitung heuristic untuk next state
        heuVal = heuristics(nextState, board)
        newNode = Node(state=nextState, parent = curNode,
move=dirChar, cost=newCost, heuristic=heuVal)
        heappush(frontier, (heuVal, newNode))
        frontier_set.add(nextState)
    return {'solution': None, 'cost': 0, 'iterations': iters, 'found':
False}

```

4.1.4. AStar.py

```

from heapq import heappush, heappop
from src.core.state import State
from src.core.node import Node
from src.core.movement import get_neighbors

def aStar(board, heuristic):
    # init state dan node
    startState = State(board.start.x, board.start.y, last_num=-1)

```

```

    startNode = Node(state=startState, parent=None, move=None, cost=0,
heuristic=heuristic(startState, board))

    # priority queue
    pq = []
    heappush(pq, (startNode.f_score, startNode))

    # nyimpen min cost utk tiap state
    visited = {}
    visited[startState] = 0

    iters = 0

    while pq:
        # pop node yg f nya paling kecil
        _, curNode = heappop(pq)
        curState = curNode.state

        iters += 1

        if curNode.cost > visited[curState]:
            continue
        # cek goal
        if board.is_goal(curState):
            return {'solution': Node.reconstruct_path(curNode), 'cost':
curNode.cost, 'iterations': iters, 'found': True}

        # expand neighbors
        neighbors = get_neighbors(board, curState)

        for dirChar, nextState, moveCost in neighbors:
            newCost = curNode.cost + moveCost
            # skip kalo udh ada path yg lebih bagus
            if nextState in visited and newCost >= visited[nextState]:
                continue

            # update visited dan pq
            visited[nextState] = newCost
            newNode = Node(state=nextState, parent=curNode, move=dirChar,
cost=newCost, heuristic=heuristic(nextState, board))

```



```

        heappush(pq, (newNode.f_score, newNode))

    return {'solution': None, 'cost': 0, 'iterations': iters, 'found':
False}

```

4.2. Folder core

4.2.1. board.py

```

class Board:
    def __init__(self, grid, costs, start, goal, max_num):
        self.grid = grid
        self.costs = costs
        self.start = start
        self.goal = goal
        self.max_num = max_num
        self.rows = len(grid)
        self.cols = len(grid[0])
        self.checkpointPos = {}
        self.goalPos = None
        for i in range(self.rows):
            for j in range(self.cols):
                tile = grid[i][j]
                if tile.isdigit():
                    self.checkpointPos[int(tile)] = (i, j)
                elif tile == 'O':
                    self.goalPos = (i, j)

    def is_inside(self, x, y):
        return 0 <= x < self.rows and 0 <= y < self.cols

    def get_tile(self, x, y):
        return self.grid[x][y]

    def is_goal(self, state):
        return self.grid[state.x][state.y] == 'O' and state.last_num ==
self.max_num

```

4.2.2. movement.py

```

from src.core.state import State

```

```

DIRECTIONS = {
    "UP": (-1, 0),
    "DOWN": (1, 0),
    "LEFT": (0, -1),
    "RIGHT": (0, 1)
}

def slide(board, state, direction):
    dx, dy = DIRECTIONS[direction]
    x, y = state.x, state.y
    current_last_num = state.last_num
    total_movement_cost = 0

    while True:
        nx, ny = x + dx, y + dy

        # game over kalo keluar dari board
        if not board.is_inside(nx, ny):
            return None

        tile = board.get_tile(nx, ny)

        # game over kalo masuk ke lava
        if tile == 'L':
            return None

        # berhenti kalo nabrak batu
        if tile == 'X':
            break

        # kalo lewat angka
        if tile.isdigit():
            # validasi urutan
            if int(tile) == current_last_num + 1:
                current_last_num = int(tile)
            elif int(tile) <= current_last_num: #angka udah pernah
dilewatin jadinya aman
                pass
            else:

```

```

        return None

    # update cost
    total_movement_cost += board.costs[nx][ny]

    # lanjut geser
    x, y = nx, ny
    #kalau ga ada pergerakan sama sekali, return None
    if x == state.x and y == state.y:
        return None
    return State(x, y, current_last_num), total_movement_cost

def get_neighbors(board, state):
    neighbors = []

    for d, (dx, dy) in DIRECTIONS.items():
        result = slide(board, state, d)
        if result is None: #kalau masuk lava/keluar map/checkpoint salah
            continue
        nextS, moveCost = result
        if nextS.x == state.x and nextS.y == state.y: #cek apakah
benar-benar bergerak dan bukan stuck in place
            continue
        dChar = d[0] #ambil char movement
        neighbors.append((dChar, nextS, moveCost))

    return neighbors

```

4.2.3. node.py

```

#search node, isinya state dan metadata buat pathfinding
class Node:
    def __init__(self, state, parent=None, move=None, cost=0,
heuristic=0):
        self.state = state
        self.parent = parent
        self.move = move
        self.cost = cost #cost cumulative dari start
        self.heuristic = heuristic #heuristic estimate ke goal

    @property

```

```

def f_score(self): #untuk A*
    return self.cost + self.heuristic

def __lt__(self, other): #buat ngecompare priority queue
    if self.f_score != other.f_score:
        return self.f_score < other.f_score
    return self.cost < other.cost

def __repr__(self):
    return f"({self.state}, {self.cost}, {self.heuristic}, {self.move})"

    @staticmethod
    def reconstruct_path(node): #reconstruct dari goal node dengan
ngikutin parent pointers
        path = []
        cur = node
        while cur.parent is not None:
            path.append(cur.move)
            cur = cur.parent
        path.reverse() #direverse karena dibangunnya backwards tadi
        return "".join(path)

    @staticmethod
    def reconstruct_states(node):
        states = []
        while node is not None:
            states.append(node.state)
            node = node.parent
        states.reverse()
        return states

```

4.2.4. state.py

```

class State:
    def __init__(self, x, y, last_num = -1):
        self.x = x
        self.y = y
        self.last_num = last_num

    def __eq__(self, other):

```

```

        if not isinstance(other, State): return False
        return self.x == other.x and self.y == other.y and self.last_num
== other.last_num

    def __hash__(self):
        return hash((self.x, self.y, self.last_num))

    def __repr__(self):
        return f"({self.x},{self.y},{self.last_num})"

```

4.3. Folder gui

4.3.1. Folder components

4.3.1.1. button.py

```

import pygame
from src.gui.colors import *
from src.gui.constants import *

class Button:
    def __init__(self, x, y, width, height, text, callback, font_size=20,
enabled=True):
        self.rect = pygame.Rect(x, y, width, height)
        self.text = text
        self.callback = callback
        self.enabled = enabled
        self.hovered = False
        self.font = pygame.font.SysFont(FONT_FAMILY,font_size,
bold=FONT_BOLD)

    def handle_event(self, event):
        if not self.enabled:
            return

        if event.type == pygame.MOUSEMOTION:
            self.hovered = self.rect.collidepoint(event.pos)
        elif event.type == pygame.MOUSEBUTTONDOWN:
            if event.button == 1:
                if self.rect.collidepoint(event.pos):
                    self.callback()

```

```

def draw(self, screen):
    if not self.enabled:
        color = PANEL_LIGHT
    elif self.hovered:
        color = ACCENT_HOVER
    else:
        color = ACCENT

    pygame.draw.rect(screen, color, self.rect,
border_radius=CORNER_RADIUS)
    text_surface = self.font.render(self.text, True, TEXT)
    text_rect = text_surface.get_rect(center=self.rect.center)
    screen.blit(text_surface, text_rect)

```

4.3.1.2. dropdown.py

```

import pygame
from src.gui.colors import *
from src.gui.constants import *

class Dropdown:
    def __init__(self, x, y, width, height, options, default_index=0,
callback=None):
        self.rect = pygame.Rect(x, y, width, height)
        self.options = options
        self.selected_index = default_index
        self.callback = callback
        self.expanded = False
        self.hovered_index = -1
        self.enabled = True
        self.font = pygame.font.SysFont(FONT_FAMILY, 18, bold=FONT_BOLD)

    @property
    def selected(self):
        return self.options[self.selected_index]

    def set_enabled(self, enabled):
        self.enabled = enabled

    def handle_event(self, event):
        if not self.enabled:

```



```

        if option_rect.collidepoint(event.pos):
            self.hovered_index = i
            break

    def draw(self, screen):
        base_color = PANEL_LIGHT if self.enabled else PANEL
        pygame.draw.rect(screen, base_color, self.rect,
border_radius=CORNER_RADIUS)
        pygame.draw.rect(screen, BORDER, self.rect, width=1,
border_radius=CORNER_RADIUS)

        text_surface = self.font.render(self.selected, True, TEXT)
        screen.blit(text_surface, (self.rect.x + 12, self.rect.y + 10))

        # arrow
        arrow = '▼' if not self.expanded else '▲'
        arrow_surface = self.font.render(arrow, True, TEXT)
        arrow_rect = arrow_surface.get_rect(
            center=(
                self.rect.right - 16,
                self.rect.centery
            )
        )
        screen.blit(arrow_surface, arrow_rect)

        # expanded options
        if self.expanded:
            for i, option in enumerate(self.options):
                option_rect = pygame.Rect(
                    self.rect.x,
                    self.rect.y + (i + 1) * self.rect.height,
                    self.rect.width,
                    self.rect.height
                )

                color = PANEL_LIGHT

                if i == self.hovered_index:
                    color = ACCENT

```



```

        pygame.draw.rect(screen, color, option_rect,
border_radius=CORNER_RADIUS)
        pygame.draw.rect(screen, BORDER, option_rect, width=1,
border_radius=CORNER_RADIUS)

        option_surface = self.font.render(option, True, TEXT)
        screen.blit(option_surface, (option_rect.x + 12,
option_rect.y + 10))

```

4.3.1.3. label.py

```

import pygame
from src.gui.colors import *
from src.gui.constants import *

class Label:
    def __init__(self, x, y, text, font_size=20, color=TEXT, bold=True):
        self.x = x
        self.y = y
        self.text = text
        self.color = color
        self.font = pygame.font.SysFont(FONT_FAMILY, font_size, bold=bold)

    def set_text(self, text):
        self.text = text

    def draw(self, screen):
        surface = self.font.render(self.text, True, self.color)
        screen.blit(surface, (self.x, self.y))

```

4.3.1.4. panel.py

```

import pygame
from src.gui.colors import *
from src.gui.constants import *

class Panel:
    def __init__(self, x, y, width, height, color=PANEL):
        self.rect = pygame.Rect(x, y, width, height)
        self.color = color

```

```

    def draw(self, screen):
        pygame.draw.rect(screen, self.color, self.rect,
border_radius=CORNER_RADIUS)
        pygame.draw.rect(screen, BORDER, self.rect, width=1,
border_radius=CORNER_RADIUS)

```

4.3.1.5. slider.py

```

import pygame
from src.gui.colors import *
from src.gui.constants import *

class Slider:
    def __init__(self, x, y, width, min_value, max_value, initial_value,
callback=None):
        self.x = x
        self.y = y
        self.width = width
        self.min_value = min_value
        self.max_value = max_value
        self.value = initial_value
        self.callback = callback
        self.dragging = False
        self.track_rect = pygame.Rect(x, y, width, 6)
        self.knob_radius = 10
        self.font = pygame.font.SysFont(FONT_FAMILY, 16, bold=FONT_BOLD)

    def get_ratio(self):
        return ((self.value - self.min_value)/(self.max_value -
self.min_value))

    def get_knob_x(self):
        return self.x + int(self.get_ratio() * self.width)

    def handle_event(self, event):
        knob_x = self.get_knob_x()
        knob_rect = pygame.Rect(
            knob_x - self.knob_radius,
            self.y - self.knob_radius,
            self.knob_radius * 2,
            self.knob_radius * 2

```

```

)

if event.type == pygame.MOUSEBUTTONDOWN:
    if event.button == 1:
        if knob_rect.collidepoint(event.pos):
            self.dragging = True

elif event.type == pygame.MOUSEBUTTONUP:
    if event.button == 1:
        self.dragging = False

elif event.type == pygame.MOUSEMOTION:
    if self.dragging:
        mouse_x = event.pos[0]
        mouse_x = max(self.x, mouse_x)
        mouse_x = min(self.x + self.width, mouse_x)

        ratio = (mouse_x - self.x) / self.width

        self.value = (
            self.min_value
            + ratio * (self.max_value - self.min_value)
        )

        if self.callback:
            self.callback(self.value)

def draw(self, screen):
    pygame.draw.rect(screen, PANEL_LIGHT, self.track_rect,
border_radius=4)
    filled_rect = pygame.Rect(self.x, self.y, self.get_knob_x() -
self.x, 6)
    pygame.draw.rect(screen, ACCENT, filled_rect, border_radius=4)
    pygame.draw.circle(screen, ACCENT_HOVER, (self.get_knob_x(),
self.y + 3), self.knob_radius)
    value_surface = self.font.render(f'{int(self.value)} ms', True,
TEXT)
    screen.blit(value_surface, (self.x, self.y - 20))

```

4.3.2. Folder services

4.3.2.1. export_services.py

```
import os
import time

class ExportService:

    @staticmethod
    def export_solution(
        board,
        result,
        steps,
        algorithm,
        heuristic=None
    ):

        # simpan ke test/output
        BASE_DIR = os.path.abspath(
            os.path.join(
                os.path.dirname(__file__),
                '../../../../../'
            )
        )
        output_dir = os.path.join(BASE_DIR, 'test', 'output')

        os.makedirs(output_dir, exist_ok=True)

        timestamp = time.strftime('%Y%m%d_%H%M%S')
        algo_name = algorithm.replace('*', 'Star')
        filename = f'solution_{algo_name}'
        if heuristic:
            filename += f'_{heuristic}'
        filename += f'_{timestamp}.txt'

        filepath = os.path.join(
            output_dir,
            filename
        )
```

```
with open(filepath, 'w') as file:
    # HEADER
    file.write('=' * 45 + '\n')
    file.write('Ice Sliding Puzzle Solution\n')
    file.write('=' * 45 + '\n\n')

    file.write(
        f'Solution Path: '
        f'{result["solution"]}\n'
    )

    file.write(
        f'Total Cost: '
        f'{result["cost"]}\n'
    )

    file.write(
        f'Iterations: '
        f'{result["iterations"]}\n'
    )

    file.write(
        f'Execution Time: '
        f'{result["time_ms"]:.2f} ms\n'
    )

    file.write('\n')

    # STEP SECTION
    file.write('=' * 45 + '\n')
    file.write('Step-by-step Solution\n')
    file.write('=' * 45 + '\n\n')

    for step_index, move, pos, last_num in steps:
        # title step
        if step_index == 0:
            file.write('Initial State:\n')
        else:
            file.write(
                f'Step {step_index}: '
```

```

        f'{move}\n'
    )

    # gambar board
    for row in range(board.rows):
        row_text = ''
        for col in range(board.cols):
            if (row, col) == pos:
                row_text += 'Z'

            else:
                row_text += (board.grid[row][col])

        file.write(row_text + '\n')

    file.write('\n')

    # FOOTER
    file.write('=' * 45 + '\n')
    file.write('End of Solution\n')
    file.write('=' * 45 + '\n')

    return os.path.abspath(filepath)

```

4.3.2.2. playback_service.py

```

import pygame

class PlaybackService:
    def __init__(self):
        self.current_step = 0
        self.playing = False
        self.speed_ms = 500
        self.last_update = 0
        self.steps = []

    def load_steps(self, steps):
        self.steps = steps
        self.current_step = 0
        self.playing = False

```

```
def toggle_play(self):
    self.playing = not self.playing
    self.last_update = pygame.time.get_ticks()

def stop(self):
    self.playing = False

def set_speed(self, speed_ms):
    self.speed_ms = speed_ms

def next_step(self):
    if not self.steps:
        return

    if self.current_step < len(self.steps) - 1:
        self.current_step += 1

def prev_step(self):
    if not self.steps:
        return

    if self.current_step > 0:
        self.current_step -= 1

def first_step(self):
    self.current_step = 0

def last_step(self):
    if self.steps:
        self.current_step = len(self.steps) - 1

def jump_to(self, step):
    if not self.steps:
        return

    step = max(0, step)
    step = min(step, len(self.steps) - 1)

    self.current_step = step
```

```

def update(self):
    if not self.playing:
        return

    if not self.steps:
        return

    now = pygame.time.get_ticks()
    if now - self.last_update >= self.speed_ms:
        self.last_update = now
        if self.current_step < len(self.steps) - 1:
            self.current_step += 1
        else:
            self.playing = False

def get_current_step(self):
    if not self.steps:
        return None
    return self.steps[self.current_step]

def get_progress_text(self):
    if not self.steps:
        return '0 / 0'

    return (f'{self.current_step}'f' / 'f'{len(self.steps)-1}')

```

4.3.2.3. solver_service.py

```

import time
from src.algo_and_heu.UCS import ucs
from src.algo_and_heu.GBFS import gbfs
from src.algo_and_heu.AStar import aStar
from src.algo_and_heu.heuristics import getHeuristic

class SolverService:
    @staticmethod
    def run(board, algorithm, heuristic_name=None):
        algorithm = algorithm.upper()
        heuristic_func = None

```



```

    if algorithm in ['GBFS', 'A*']:
        heuristic_func = getHeuristic(heuristic_name.upper())
        if heuristic_func is None:
            raise ValueError(f'Invalid heuristic: {heuristic_name}')

    start_time = time.perf_counter()

    if algorithm == 'UCS':
        result = ucs(board)
    elif algorithm == 'GBFS':
        result = gbfs(board, heuristic_func)
    elif algorithm == 'A*':
        result = aStar(board, heuristic_func)
    else:
        raise ValueError(f'Invalid algorithm: {algorithm}')

    end_time = time.perf_counter()
    execution_ms = (end_time - start_time) * 1000
    result['time_ms'] = execution_ms
    return result

```

4.3.3. Folder utils

4.3.3.1. board_steps.py

```

from src.core.movement import DIRECTIONS

def generate_solution_steps(board, solution):
    steps = []
    current_x = board.start.x
    current_y = board.start.y
    last_checkpoint = -1

    steps.append((0, 'Initial', (current_x, current_y), last_checkpoint))

    direction_map = {
        'U': 'UP',
        'D': 'DOWN',
        'L': 'LEFT',
        'R': 'RIGHT'
    }

```

```

for step_index, move_char in enumerate(solution, start=1):
    direction = direction_map[move_char]
    dx, dy = DIRECTIONS[direction]

    while True:
        nx = current_x + dx
        ny = current_y + dy

        if not board.is_inside(nx, ny):
            break

        tile = board.get_tile(nx, ny)

        if tile == 'X':
            break

        if tile.isdigit():
            checkpoint_num = int(tile)
            if checkpoint_num == last_checkpoint + 1:
                last_checkpoint = checkpoint_num

        current_x = nx
        current_y = ny

        steps.append((step_index, move_char, (current_x, current_y),
last_checkpoint))

    return steps

```

4.3.3.2. timer.py

```

import time

class Timer:
    def __init__(self):
        self.start_time = 0
        self.end_time = 0

    def start(self):
        self.start_time = time.perf_counter()

```

```
def stop(self):
    self.end_time = time.perf_counter()

def elapsed_ms(self):
    return (self.end_time - self.start_time) * 1000
```

4.3.4. Folder views

4.3.4.1. board_view.py

```
import pygame
from src.gui.colors import *
from src.gui.constants import *

class BoardView:
    def __init__(self, app):
        self.app = app
        self.tile_size = 50

    def get_tile_color(self, tile):
        if tile == 'X':
            return WALL

        if tile == 'L':
            return DANGER

        if tile == 'O':
            return GOAL

        if tile.isdigit():
            return CHECKPOINT

        return BOARD_EMPTY

    def draw(self, screen):
        board = self.app.board

        if board is None:
            return

        rows = board.rows
        cols = board.cols
```



```

        if tile != 'X' and tile != 'L':
            cost = str(board.costs[row][col])

            cost_font = pygame.font.SysFont(
                FONT_FAMILY,
                max(12, self.tile_size // 6),
                bold=True
            )

            cost_surface = cost_font.render(
                cost,
                True,
                TEXT_DIM
            )

            cost_rect = cost_surface.get_rect(
                bottomright=(
                    rect.right - 5,
                    rect.bottom - 3
                )
            )

            screen.blit(cost_surface, cost_rect)

    player_rect = pygame.Rect(
        start_x + player_pos[1] * self.tile_size,
        start_y + player_pos[0] * self.tile_size,
        self.tile_size,
        self.tile_size
    )

    pygame.draw.circle(
        screen,
        PLAYER,
        player_rect.center,
        self.tile_size // 3
    )

```

4.3.4.2. playback_view.py

```
from src.gui.components.button import Button
from src.gui.components.slider import Slider
from src.gui.components.panel import Panel
from src.gui.components.label import Label
from src.gui.constants import *

class PlaybackView:
    def __init__(self, app):
        self.app = app
        self.panel = Panel(320, 620, 930, 90)
        self.components = []
        self.build()

    def build(self):
        self.components.clear()
        x = 340
        first_btn = Button(x, 640, 50, 40, '<<',
self.app.playback.first_step)

        x += 60
        prev_btn = Button(x, 640, 50, 40, '<',self.app.playback.prev_step)

        x += 60
        play_btn = Button(x, 640, 70, 40, 'PLAY',
self.app.playback.toggle_play)
        self.play_button = play_btn

        x += 80
        next_btn = Button(x, 640, 50, 40, '>', self.app.playback.next_step)

        x += 60
        last_btn = Button(x, 640, 50, 40, '>>',
self.app.playback.last_step)

        x += 100
        slider = Slider(x, 640 + 18, 220, 100, 2000, 500,
self.change_speed)
        self.slider = slider
```

```

        self.components.extend([first_btn, prev_btn, play_btn, next_btn,
last_btn, slider])

    def change_speed(self, value):
        self.app.playback.set_speed(value)

    def update(self):
        if self.app.playback.playing:
            self.play_button.text = 'PAUSE'
        else:
            self.play_button.text = 'PLAY'

    def handle_event(self, event):
        for component in self.components:
            if hasattr(component, 'handle_event'):
                component.handle_event(event)

    def draw(self, screen):
        self.panel.draw(screen)
        self.update()

        for component in self.components:
            component.draw(screen)

        Label(1090, 640, self.app.playback.get_progress_text(),
18).draw(screen)

```

4.3.4.3. sidebar_view.py

```

import pygame

from src.gui.components.button import Button
from src.gui.components.dropdown import Dropdown
from src.gui.components.label import Label
from src.gui.components.panel import Panel
from src.gui.constants import *
from src.gui.colors import *

class SidebarView:
    def __init__(self, app):
        self.app = app

```

```

self.width = 300
self.panel = Panel(0, 0, self.width, WINDOW_HEIGHT, color=SIDEBAR)
self.components = []
self.build()

def build(self):
    self.components.clear()

    y = 30
    title = Label(30, y, "Ice Sliding Solver", font_size=28)
    self.components.append(title)

    y += 70
    load_button = Button(30, y, 240, 45, "LOAD MAP",
self.app.load_map)
    self.components.append(load_button)

    y += 80
    algo_label = Label(30, y, "Algorithm", font_size=18)
    self.components.append(algo_label)

    y += 30
    algo_dropdown = Dropdown(30, y, 240, 42, ['UCS', 'GBFS', 'A*'],
callback=self.app.set_algorithm)
    self.components.append(algo_dropdown)
    self.algo_dropdown = algo_dropdown

    y += 80
    heuristic_label = Label(30, y, "Heuristic", font_size=18)
    self.components.append(heuristic_label)

    y += 30
    heuristic_dropdown = Dropdown(30, y, 240, 42, ['H1', 'H2', 'H3'],
callback=self.app.set_heuristic)
    self.components.append(heuristic_dropdown)
    self.heuristic_dropdown = heuristic_dropdown

    y += 90
    solve_button = Button(30, y, 240, 50, "RUN SOLVER",
self.app.run_solver)

```



```

        self.components.append(solve_button)

    def handle_event(self, event):
        for component in self.components:
            if hasattr(component, 'handle_event'):
                component.handle_event(event)

    def draw(self, screen):
        self.panel.draw(screen)
        expanded_dropdown = None

        # draw semua selain dropdown expanded
        for component in self.components:
            if isinstance(component, Dropdown):
                if component.expanded:
                    expanded_dropdown = component
                    continue
            component.draw(screen)

        # draw dropdown expanded paling akhir
        if expanded_dropdown:
            expanded_dropdown.draw(screen)

```

4.3.4.4. statistics_view.py

```

from src.gui.components.label import Label
from src.gui.components.panel import Panel
from src.gui.constants import *
from src.gui.colors import *

class StatisticsView:
    def __init__(self, app):
        self.app = app
        self.panel = Panel(20, 500, 260, 170)

    def draw(self, screen):
        self.panel.draw(screen)
        result = self.app.result

        Label(40, 510, 'STATISTICS', 24).draw(screen)

```

```

        if result is None:
            Label(40, 550, "No solution loaded.", 16,
TEXT_DIM).draw(screen)
            return

            Label(40, 550, f'Path: {result["solution"]}', 16,
bold=False).draw(screen)
            Label(40, 575, f'Cost: {result["cost"]}', 16,
bold=False).draw(screen)
            Label(40, 600, f'Iterations: {result["iterations"]}', 16,
bold=False).draw(screen)
            Label(40, 625, f'Time: {result["time_ms"]:.2f} ms', 16,
bold=False).draw(screen)

```

4.3.5. app.py

```

import threading
import pygame
import tkinter as tk

from tkinter import filedialog

from src.parser.parser import parse_file

from src.gui.constants import *
from src.gui.colors import *

from src.gui.views.sidebar_view import SidebarView
from src.gui.views.board_view import BoardView
from src.gui.views.statistics_view import StatisticsView
from src.gui.views.playback_view import PlaybackView

from src.gui.services.playback_service import PlaybackService
from src.gui.services.solver_service import SolverService
from src.gui.services.export_service import ExportService

from src.gui.utils.board_steps import generate_solution_steps

```

```

class App:
    def __init__(self):
        pygame.init()
        self.screen = pygame.display.set_mode((WINDOW_WIDTH,
WINDOW_HEIGHT))

        pygame.display.set_caption('Ice Sliding Puzzle Solver')

        self.clock = pygame.time.Clock()

        # ===== DATA =====
        self.board = None
        self.algorithm = 'UCS'
        self.heuristic = 'H1'
        self.result = None
        self.solving = False

        # ===== PLAYBACK =====
        self.playback = PlaybackService()

        # ===== VIEWS =====
        self.sidebar_view = SidebarView(self)
        self.board_view = BoardView(self)
        self.statistics_view = StatisticsView(self)
        self.playback_view = PlaybackView(self)

        # =====
        # CONFIG
        # =====

    def set_algorithm(self, algorithm):
        self.algorithm = algorithm

    def set_heuristic(self, heuristic):
        self.heuristic = heuristic

        # =====
        # LOAD MAP
        # =====

```

```

def load_map(self):
    root = tk.Tk()
    root.withdraw()
    filepath = filedialog.askopenfilename(filetypes=[('Text Files',
'*.txt')])

    root.destroy()

    if not filepath:
        return

    try:
        self.board = parse_file(filepath)
        self.result = None
        self.playback.load_steps([])
        print('Map loaded.')

    except Exception as error:
        print(error)

# =====
# SOLVER
# =====

def run_solver(self):
    if self.board is None:
        print('No board loaded.')
        return

    if self.solving:
        return

    self.solving = True

    solver_thread = threading.Thread(target=self.solve_thread,
daemon=True)
    solver_thread.start()

def solve_thread(self):
    try:

```

```

        result = SolverService.run(
            self.board,
            self.algorithm,
            self.heuristic
        )

        self.result = result

        if result['found']:
            steps = generate_solution_steps(self.board,
result['solution'])
            self.playback.load_steps(steps)

            output_path = ExportService.export_solution(
                board=self.board,
                result=result,
                steps=steps,
                algorithm=self.algorithm,
                heuristic=self.heuristic
            )

            print(f'Solution exported:\n{output_path}')

        else:
            self.playback.load_steps([])

    except Exception as error:
        print(error)

    self.solving = False

# =====
# EVENTS
# =====

def handle_events(self):
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            pygame.quit()
            raise SystemExit

```

```

        self.sidebar_view.handle_event(event)
        self.playback_view.handle_event(event)

    # keyboard playback
    if event.type == pygame.KEYDOWN:
        if event.key == pygame.K_RIGHT:
            self.playback.next_step()
        elif event.key == pygame.K_LEFT:
            self.playback.prev_step()
        elif event.key == pygame.K_SPACE:
            self.playback.toggle_play()

# =====
# UPDATE
# =====

def update(self):
    self.playback.update()

# =====
# DRAW
# =====

def draw(self):
    self.screen.fill(BACKGROUND)
    self.sidebar_view.draw(self.screen)
    self.board_view.draw(self.screen)
    self.statistics_view.draw(self.screen)
    self.playback_view.draw(self.screen)

    if self.solving:
        font = pygame.font.SysFont(FONT_FAMILY, 26, bold=FONT_BOLD)
        text_surface = font.render('Solving...', True, TEXT)
        self.screen.blit(text_surface, (520, 250))

    pygame.display.flip()

# =====
# MAIN LOOP

```

```
# =====

def run(self):
    while True:
        self.handle_events()
        self.update()
        self.draw()
        self.clock.tick(FPS)
```

4.3.6. colors.py

```
BACKGROUND = (18, 19, 24)

SIDEBAR = (33, 34, 40)

PANEL = (39, 40, 48)
PANEL_LIGHT = (56, 57, 68)

BORDER = (60, 62, 74)

TEXT = (245, 245, 250)
TEXT_DIM = (145, 148, 165)

ACCENT = (24, 132, 255)
ACCENT_HOVER = (45, 150, 255)

SUCCESS = (60, 200, 100)

WARNING = (255, 210, 40)

DANGER = (255, 60, 60)

BOARD_EMPTY = (42, 43, 50)

PLAYER = (60, 210, 90)

CHECKPOINT = (24, 132, 255)

GOAL = (255, 210, 40)

WALL = (74, 75, 86)
```

4.3.7. constants.py

```
WINDOW_WIDTH = 1280
WINDOW_HEIGHT = 720

FPS = 60

DEFAULT_FONT = 'arial'
FONT_FAMILY = 'Segoe UI'
FONT_BOLD = True

BUTTON_HEIGHT = 42
DROPDOWN_HEIGHT = 42
SLIDER_HEIGHT = 28

PADDING = 12
CORNER_RADIUS = 10
```

4.4. Folder parser

4.4.1. parser.py

```
from src.core.board import Board
from src.core.state import State

def parse_file(filepath):
    grid = []
    costs = []
    start = None
    goal = None
    numbers = []

    with open(filepath, 'r') as f:
        # baca N dan M
        line = f.readline().strip()
        if not line:
            raise ValueError("File kosong")

        try:
            n, m = map(int, line.split())
        except ValueError:
```



```

        raise ValueError("Baris pertama harus berisi dua angka (N M)")

# baca grid map
for i in range(n):
    row_str = f.readline().strip()

    if len(row_str) != m:
        raise ValueError(f"Baris map ke-{i+1} panjangnya {len(row_str)}, harusnya {m}")

    row_list = list(row_str)
    grid.append(row_list)

    for j, cell in enumerate(row_list):
        if cell == 'Z':
            start = State(i, j)
        elif cell == 'O':
            goal = State(i, j)
        elif cell.isdigit():
            if int(cell) < 0 or int(cell) > 9:
                raise ValueError(f"Angka harus antara 0-9: {cell}")
            else:
                numbers.append(int(cell))
        elif cell not in ['X', 'L', '*']:
            raise ValueError(f"Karakter tidak valid di grid: {cell}")

# validasi urutan angka
if numbers:
    numbers.sort()
    for idx, num in enumerate(numbers):
        if num != idx:
            raise ValueError(f"Angka harus berurutan dari 0")
    max_num = numbers[-1]
else:
    max_num = -1

# baca costs
for i in range(n):

```

```

        cost_line = f.readline().strip()
        if not cost_line:
            raise ValueError(f"Baris cost ke-{i+1} tidak ditemukan
(harus ada {n} baris)")

        cost_row = list(map(int, cost_line.split()))
        if len(cost_row) != m:
            raise ValueError(f"Baris cost ke-{i+1} punya
{len(cost_row)} kolom, harusnya {m}")

        for cost in cost_row:
            if cost < 0:
                raise ValueError(f"Cost tidak boleh negatif pada baris
{i+1}: {cost}")
        costs.append(cost_row)

    if start is None or goal is None:
        raise ValueError("Map harus punya Start (Z) dan Goal (O)")

    return Board(grid, costs, start, goal, max_num)

```

4.5. main.py

```

import os
import sys
import time
from src.parser.parser import parse_file
from src.algo_and_heu.UCS import ucs
from src.algo_and_heu.GBFS import gbfs
from src.algo_and_heu.AStar import aStar
from src.algo_and_heu.heuristics import getHeuristic
from src.core.movement import DIRECTIONS

try:
    if os.name == 'nt': #windows
        import msvcrt
    else: #unix/linux/mac
        import tty
        import termios
except ImportError:

```

```

pass

def clearScreen():
    os.system('cls' if os.name == 'nt' else 'clear')

def displayBoard(board, playerPos):
    print()
    for i in range(board.rows):
        rowStr = ""
        for j in range(board.cols):
            if(i, j) == playerPos:
                rowStr += "Z" #player pos
            else:
                rowStr += board.grid[i][j]
        print(rowStr)
    print()

def getSolutionSteps(board, solution):
    steps = []
    currentX, currentY = board.start.x, board.start.y
    #initial state
    steps.append((0, "Initial", (currentX, currentY)))
    #simulasi tiap move
    for numStep, dirChar in enumerate(solution, start=1):
        #map char ke nama direction full
        directionMap = {'U': 'UP', 'D': 'DOWN', 'L': 'LEFT', 'R': 'RIGHT'}
        dir = directionMap[dirChar]
        dx, dy = DIRECTIONS[dir]
        #slide sampai berhenti
        while True:
            nx, ny = currentX + dx, currentY + dy
            if not board.is_inside(nx, ny): break
            tile = board.get_tile(nx, ny)
            if tile == 'X': break
            currentX, currentY = nx, ny
        steps.append((numStep, dirChar, (currentX, currentY)))
    return steps

def playback(board, steps):
    curStep = 0

```

```

maxStep = len(steps)-1
def displayCurStep():
    clearScreen()
    stepNum, dir, pos = steps[curStep]
    if stepNum == 0:
        print("="*45)
        print("Initial State")
        print("="*45)
    else:
        print("="*45)
        print(f"Step {stepNum}: {dir}")
        print("="*45)
    displayBoard(board, pos)
    print("="*45)
    print(f"Step {curStep}/{maxStep}")
    print("\nControls:")
    print("right arrow: next step")
    print("left arrow: previous step")
    print("Q: quit playback")
    print("ESC: jump to specific step")
    print("="*45)
displayCurStep()
# Platform-specific keyboard handling
if os.name == 'nt': #windows
    while True:
        if msvcrt.kbhit():
            key = msvcrt.getch()
            if key == b'\xe0': #arrow key prefix
                key = msvcrt.getch()
                if key == b'K': #left arrow
                    if curStep > 0:
                        curStep -= 1
                        displayCurStep()
                elif key == b'M': # Right arrow
                    if curStep < maxStep:
                        curStep += 1
                        displayCurStep()
            elif key == b'\x1b': #ESC
                print("\n>> Pada step berapa anda ingin melakukan
playback :")

```

```

        try:
            jump_to = int(input().strip())
            if 0 <= jump_to <= maxStep:
                curStep = jump_to
                displayCurStep()
            else:
                print(f"Step harus berada di antara 0 dan
{maxStep}")

                time.sleep(1)
                displayCurStep()
        except ValueError:
            print("Invalid input")
            time.sleep(1)
            displayCurStep()
        elif key.lower() == b'q': #Quit
            break
    else: #unix/linux/mac
        fd = sys.stdin.fileno()
        old_settings = termios.tcgetattr(fd)
        try:
            tty.setraw(sys.stdin.fileno())
            while True:
                key = sys.stdin.read(1)
                if key == '\x1b': #escape sequence
                    next_key = sys.stdin.read(1)
                    if next_key == '[':
                        arrow = sys.stdin.read(1)
                        if arrow == 'D': #left arrow
                            if curStep > 0:
                                curStep -= 1
                                termios.tcsetattr(fd, termios.TCSADRAIN,
old_settings)

                                displayCurStep()
                                tty.setraw(sys.stdin.fileno())
                            elif arrow == 'C': #right arrow
                                if curStep < maxStep:
                                    curStep += 1
                                    termios.tcsetattr(fd, termios.TCSADRAIN,
old_settings)

                                    displayCurStep()

```

```

        tty.setraw(sys.stdin.fileno())
    else:
        #ESC key alone
        termios.tcsetattr(fd, termios.TCSADRAIN,
old_settings)

        print("\n>> Pada step berapa anda ingin melakukan
playback :")

        try:
            jump_to = int(input().strip())
            if 0 <= jump_to <= maxStep:
                curStep = jump_to
                displayCurStep()
            else:
                print(f"Step harus berada di antara 0 dan
{maxStep}")

                time.sleep(1)
                displayCurStep()
        except ValueError:
            print("Invalid input")
            time.sleep(1)
            displayCurStep()

        tty.setraw(sys.stdin.fileno())
    elif key.lower() == 'q':
        break

    finally:
        termios.tcsetattr(fd, termios.TCSADRAIN, old_settings)
clearScreen()

def simplePlayback(board, steps, startStep=0):
    for i in range(startStep, len(steps)):
        clearScreen()
        stepNum, dir, pos = steps[i]
        if stepNum == 0:
            print("="*45)
            print("Initial State")
            print("="*45)
        else:
            print("="*45)
            print(f"Step {stepNum}: {dir}")
            print("="*45)

```

```

        displayBoard(board, pos)
        print(f"\nStep {i}/{len(steps)-1}")
        print("Press ENTER to continue...")
        input()

def saveSolution(board, res, steps, filepath):
    with open(filepath, 'w') as f:
        f.write("="*45 + "\n")
        f.write("Ice Sliding Puzzle Solution\n")
        f.write("="*45 + "\n\n")
        f.write(f"Solution Path: {res['solution']}\n")
        f.write(f"Total Cost: {res['cost']}\n")
        f.write(f"Iterations: {res['iterations']}\n\n")
        f.write("="*45 + "\n")
        f.write("Step-by-step Solution\n")
        f.write("="*45 + "\n\n")
        for stepNum, dir, pos in steps:
            if stepNum == 0:
                f.write(f"Initial State:\n")
            else:
                f.write(f"Step {stepNum}: {dir}\n")
            #write state boardnya
            for i in range(board.rows):
                rowStr = ""
                for j in range(board.cols):
                    if (i, j) == pos:
                        rowStr += "Z"
                    else:
                        rowStr += board.grid[i][j]
                f.write(rowStr + "\n")
            f.write("\n")
        f.write("="*45 + "\n")
        f.write("End of Solution\n")
        f.write("="*45 + "\n")

def main():
    #minta input file
    print(">> Masukkan file input :")
    file_path = input().strip()
    try:

```

```

    board = parse_file(file_path)
    print(f"Board loaded: {board.rows}x{board.cols}")
except Exception as e:
    print(f"Error loading file: {e}")
    return

#pilih algoritma
print("\n>> Algoritma apa yang anda pilih? (UCS/GBFS/A*)")
algo = input().strip().upper()
if algo not in ['UCS', 'GBFS', 'A*']:
    print("Invalid choice")
    return

#pilih heuristic jika GBFS atau A*
heuristic = None
heuChoice = None
if algo in ['GBFS', 'A*']:
    print("\n>> Heuristic apa yang anda pilih? (H1/H2/H3)")
    heuChoice = input().strip().upper()
    heuristic = getHeuristic(heuChoice)
    if heuristic is None:
        print("Invalid choice")
        return

#jalankan algoritma
print("\n>> Mencari solusi...")
startTime = time.time()
if algo == 'UCS':
    result = ucs(board)
elif algo == 'GBFS':
    result = gbfs(board, heuristic)
elif algo == 'A*':
    result = aStar(board, heuristic)
endTime = time.time()
execTime = (endTime - startTime) * 1000 #convert ke ms
#display hasil
if not result['found']:
    print("\n" + "="*45)
    print("Tidak ada solusi yang ditemukan")
    print(f"Waktu eksekusi: {execTime:.2f} ms")
    print(f"Node yang dieksplorasi: {result['iterations']}")
    print("="*45)
    return

```



```

#generate step-by-step solution
steps = getSolutionSteps(board, result['solution'])
#tampilkan hasil
print("\n" + "="*45)
print(f"Solusi Yang Ditemukan : {result['solution']}")
print(f"Cost dari Solusi : {result['cost']}")
print("="*45)
#display semua steps
for stepNum, dir, pos in steps:
    if stepNum == 0:
        print("\nInitial")
    else:
        print(f"\nStep {stepNum} : {dir}")

    displayBoard(board, pos)
print("="*45)
print(f">> Waktu eksekusi: {execTime:.2f} ms")
print(f">> Banyak iterasi yang dilakukan: {result['iterations']}
iterasi")
print("="*45)
#opsi playback
print("\n>> Apakah Anda ingin melakukan playback? (Ya/Tidak) :")
playbackChoice = input().strip().lower()
if playbackChoice in ['ya', 'y', 'yes']:
    print("\n>> Pada step berapa anda ingin melakukan playback :")
    try:
        startStep = int(input().strip())
        if 0 <= startStep < len(steps):
            #coba masukan ke mode playback interaktif
            try:
                playback(board, steps)
            except:
                # Fallback to simple playback
                print("\nInteractive mode not supported, using simple
playback...")

                time.sleep(1)
                simplePlayback(board, steps, startStep)
        else:
            print(f"Step harus berada di antara 0 dan {len(steps)-1}")
    except ValueError:

```

```

        print("Invalid input, skipping playback")
    #opsi save solusi
    print("\n>> Apakah Anda ingin menyimpan solusi? (Ya/Tidak) :")
    saveChoice = input().strip().lower()
    if saveChoice in ['ya', 'y', 'yes']:
        #bikin output directory kalau ga exist
        output_dir = "output"
        os.makedirs(output_dir, exist_ok=True)
        #generate filename
        timestamp = time.strftime("%Y%m%d_%H%M%S")
        if algo == 'A*':
            safe_algo = 'AStar'
        else:
            safe_algo = algo
        outputfilename =
f"{output_dir}/solution_{safe_algo}_{timestamp}.txt"
        if heuChoice:
            outputfilename =
f"{output_dir}/solution_{safe_algo}_{heuChoice}_{timestamp}.txt"
        saveSolution(board, result, steps, outputfilename)
        #ambil path absolut
        absPath = os.path.abspath(outputfilename)
        print(f"\n>> Solusi disimpan pada {absPath}")
    print("\nProgram selesai")

if __name__ == "__main__":
    main()

```

4.6. run_gui.py

```

from src.gui.app import App

if __name__ == "__main__":
    game = App()
    game.run()

```

BAB 5 PENGUJIAN

5.1. Pengujian

Input txt yang diujikan yaitu sebagai berikut:

```
7 7
XXXXXXX
X0***X
X**X**X
X***OX
X1***LX
XZ**X*X
XXXXXXX
999 999 999 999 999 999 999
999 3 5 2 8 1 999
999 7 4 999 6 9 999
999 2 8 3 5 4 999
999 6 1 7 2 999 999
999 9 3 4 999 8 999
999 999 999 999 999 999 999
```

5.1.1. Testcase 1

Ice Sliding Solver

LOAD MAP

Algorithm
UCS ▼

Heuristic
H1 ▼

RUN SOLVER

STATISTICS
Path: RULUDRUR
Cost: 87
Iterations: 15
Time: 0.18 ms

	0		5	2	8	1
		7	4		6	9
		2	8	3	5	
	1		1	7	2	
			3	4		8

<< < PLAY > >>

500 ms

0 / 8

5.1.2. Testcase 2

Ice Sliding Solver

LOAD MAP

Algorithm
GBFS

Heuristic
H1

RUN SOLVER

STATISTICS

Path: RULUDRUR
Cost: 87
Iterations: 9
Time: 0.13 ms

	0	5	2	8	1	
	7	4		6	9	
	2	8	3	5		
1		1	7	2		
		3	4		8	

<< < PLAY > >> 500 ms 0 / 8

5.1.3. Testcase 3

Ice Sliding Solver

LOAD MAP

Algorithm
GBFS

Heuristic
H2

RUN SOLVER

STATISTICS

Path: RULUDRUR
Cost: 87
Iterations: 11
Time: 0.11 ms

	0	5	2	8	1	
	7	4		6	9	
	2	8	3	5		
1		1	7	2		
		3	4		8	

<< < PLAY > >> 500 ms 0 / 8

5.1.4. Testcase 4

Ice Sliding Solver

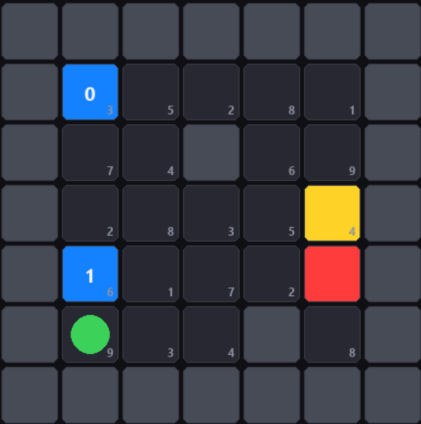
LOAD MAP

Algorithm
GBFS ▼

Heuristic
H3 ▼

RUN SOLVER

STATISTICS
Path: RULUDRUR
Cost: 87
Iterations: 11
Time: 0.18 ms



A 7x7 grid representing an ice sliding puzzle. The grid contains numbers 1-9 and some colored cells. The start cell is blue with '0' at (1,2). The goal cell is red at (4,6). A green circle is at (5,2). The path RULUDRUR is highlighted in blue.

	0	5	2	8	1	
	7	4		6	9	
	2	8	3	5	4	
	1	1	7	2		
	9	3	4		8	

<< < PLAY > >> 500 ms 0 / 8

5.1.5. Testcase 5

Ice Sliding Solver

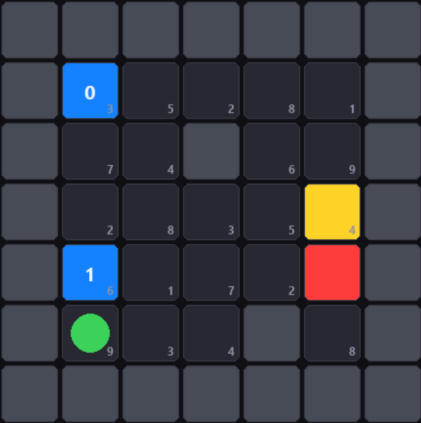
LOAD MAP

Algorithm
A* ▼

Heuristic
H1 ▼

RUN SOLVER

STATISTICS
Path: RULUDRUR
Cost: 87
Iterations: 13
Time: 0.14 ms



A 7x7 grid representing an ice sliding puzzle. The grid contains numbers 1-9 and some colored cells. The start cell is blue with '0' at (1,2). The goal cell is red at (4,6). A green circle is at (5,2). The path RULUDRUR is highlighted in blue.

	0	5	2	8	1	
	7	4		6	9	
	2	8	3	5	4	
	1	1	7	2		
	9	3	4		8	

<< < PLAY > >> 500 ms 0 / 8

5.1.6. Testcase 6

Ice Sliding Solver

LOAD MAP

Algorithm
A* ▼

Heuristic
H2 ▼

RUN SOLVER

STATISTICS

Path: RULUDRUR
Cost: 87
Iterations: 13
Time: 0.18 ms

7x7 grid map with numbers and obstacles. The green circle is at (4,1). The blue square with '0' is at (2,2). The blue square with '1' is at (4,2). The yellow square is at (3,6). The red square is at (4,6).

<< < PLAY > >>

500 ms

0 / 8

5.1.7. Testcase 7

Ice Sliding Solver

LOAD MAP

Algorithm
A* ▼

Heuristic
H3 ▼

RUN SOLVER

STATISTICS

Path: RULUDRUR
Cost: 87
Iterations: 13
Time: 0.12 ms

7x7 grid map with numbers and obstacles. The green circle is at (4,1). The blue square with '0' is at (2,2). The blue square with '1' is at (4,2). The yellow square is at (3,6). The red square is at (4,6).

<< < PLAY > >>

500 ms

0 / 8

5.2. Analisis Hasil

Berdasarkan hasil pengujian di atas, seluruh algoritma berhasil menemukan solusi yang sama, yaitu jalur RULUDRUR dengan total cost sebesar 87.

Dari sisi performa, algoritma UCS membutuhkan iterasi paling banyak, yaitu 15 iterasi, karena UCS melakukan pencarian berdasarkan cost tanpa mempertimbangkan arah menuju goal. Sebaliknya, A* mampu mengurangi jumlah iterasi menjadi 13 dengan bantuan heuristic sehingga pencarian menjadi lebih terarah.

Pada pengujian ini, GBFS menghasilkan jumlah iterasi paling sedikit. Heuristic H1 memberikan hasil terbaik dengan hanya 9 iterasi, sedangkan H2 dan H3 membutuhkan sekitar 11 iterasi. Hal ini terjadi karena H1 memperhitungkan total jarak ke seluruh checkpoint yang tersisa hingga goal sehingga estimasi yang diberikan lebih informatif dibanding heuristic lain yang hanya melihat target aktif berikutnya.

Walaupun GBFS pada kasus ini berhasil menemukan solusi optimal, secara umum algoritma tersebut tidak menjamin optimalitas karena hanya berfokus pada nilai heuristic dan mengabaikan cost perjalanan yang sudah ditempuh. Sebaliknya, UCS dan A* tetap menjamin solusi optimal, dengan A* memiliki performa yang lebih efisien dibanding UCS.

Secara keseluruhan, A* memberikan hasil yang paling seimbang karena tetap optimal namun dengan jumlah eksplorasi yang lebih sedikit dibanding UCS.

LAMPIRAN

Tabel Checklist Spesifikasi Program

No.	Poin	Ya	Tidak
1.	Program berhasil di kompilasi tanpa kesalahan	✓	
2.	Program berhasil dijalankan	✓	
3.	Solusi yang diberikan program benar dan mematuhi aturan permainan	✓	
4.	Program dapat membaca masukan berkas .txt serta menyimpan solusi dalam berkas .txt	✓	
5.	Program memiliki Graphical User Interface (GUI)	✓	
6.	Ada algoritma pathfinding alternatif selain dari spesifikasi wajib (Algoritma X dan Algoritma Y)		✓
7.	Ada tambahkan dua alternatif heuristik selain yang utama (Heuristik X dan Heuristik Y)	✓	

Repository Program

Repository program dapat dibuka melalui link berikut:

https://github.com/Beeziebloop/Tucil3_13524128_13524135

Tugas ini disusun sepenuhnya tanpa bantuan kecerdasan buatan (*Generative AI*), melainkan hasil pemikiran dan analisis mandiri.



Varistha Devi



Safira Berlianti

